NBA Advanced Statistics - MapReduce

Vivek Shah

DS 644 - 852

April 12th, 2026

Table of Contents

1. Dataset Introduction....3
1.1 Dataset Name & Source....3
1.2 Dataset Description....3
1.3 Dataset Size & Format....4
1.4 Reason for Selection....4
2. Environment Setup Screenshots & Explanations....5
2.1 Screenshot: AWS Console - 4 Ubuntu EC2 Instances Running....5
2.2 Screenshot: Java / Hadoop Installation & Hadoop Cluster....6
2.3 Screenshot: Hadoop Cluster - Fully-Distributed Mode....7
3. MapReduce Code....9
3.1 Approach....9
3.2 Code Explanation....9
3.3 Compilation & Execution Commands....15
4. Code Execution & Output Interpretation....17
4.1 MapReduce Job Completion....17
5. Challenges & Troubleshooting....18
5.1 Challenges Faced....18
5.2 Troubleshooting Steps....19
5.3 Performance Observations....20
6. Summary & Key Learnings....20
6.1 Project Reflection....20
6.2 Real-World Application....21
6.3 Future Improvements....21

Note: Sections 1 & 2 are Milestone 1. **Sections 3-6 are Milestone 2**.

My submission to Canvas includes:

- Final Project Report (Word Doc)
- NBAAdvancedMetrics (JAVA File)
- nba_results (Text File - Job Output)

1. Dataset Introduction

1.1 Dataset Name & Source

Name: NBA games data

Link: https://www.kaggle.com/datasets/nathanlauga/nba-games

1.2 Dataset Description

The “NBA Games” dataset is a multi-file, structured collection of historical NBA game and player performance data. The dataset spans from the 2004 season to the 2020 season. The dataset consists of 5 interrelated CSV files: games.csv, games_details.csv, players.csv, teams.csv, and ranking.csv. Here is a brief description of each CSV file:

- games.csv - This file contains one record per game from the 2004 season onwards, including game Id, date, and game outcome. Each row represents a unique game event. There is a “GAME_ID” which is a unique ID for each game as well as “GAME_DATE_EST” and “SEASON”. Each record has a home and away team identifiers, “HOME_TEAM_ID” and “VISITOR_TEAM_ID”. The record also captures team-level performance metrics (FG %, REB, AST, FT %, etc.) for both the home and away team.
- games_details.csv - This file contains one record per player per game, capturing individual box score statistics for each player. Each record is identified by a “GAME_ID” and “PLAYER_ID”. Besides the ID fields, each record includes minutes played, field goals made / attempted, free throws, rebounds, assists, and many other major statistical categories. This file will be used as the primary input file for the MapReduce job.
- players.csv - This file contains biographical data and identifying information for each player in the dataset, including player name and player ID. It serves as a player reference table for the entire dataset. This file will be used to map player IDs to actual names in the final output.
- ranking.csv - This file contains daily team standings records for each season, including wins, losses, home / road record, and conference ranking. For each record the dataset contains a team’s cumulative standing as of a specific date, including total wins/losses and win percentage.
- teams.csv - This file contains static identifying and descriptive information for each NBA franchise including ID, abbreviation, city and full team name. This file acts as a reference table throughout the processing pipeline, enabling the resolution of numeric team identifiers in games.csv, games_details.csv, and ranking.csv into recognizable franchise names. This file functions as a lookup table throughout the data processing pipeline, enabling the resolution of numeric team identifiers in games.csv, games_details.csv, and ranking.csv into recognizable franchise names.

All CSV files here are structured datasets, in particular, the games_details.csv is the primary file for this project. It contains the granular player box score statistics that will be needed in the MapReduce calculations. The combination of these fields provide the foundation for computing advanced performance metrics like Player Efficiency Rating (PER), Win Shares (WS), and Estimated Plus/Minus (EPM) for each player across each season.

1.3 Dataset Size & Format

The dataset files are all CSV file-format, with the primary file, games_details.csv, containing approximately 660K+ rows of player-game records. Below you can find a table outlining the size and format of each of the files.

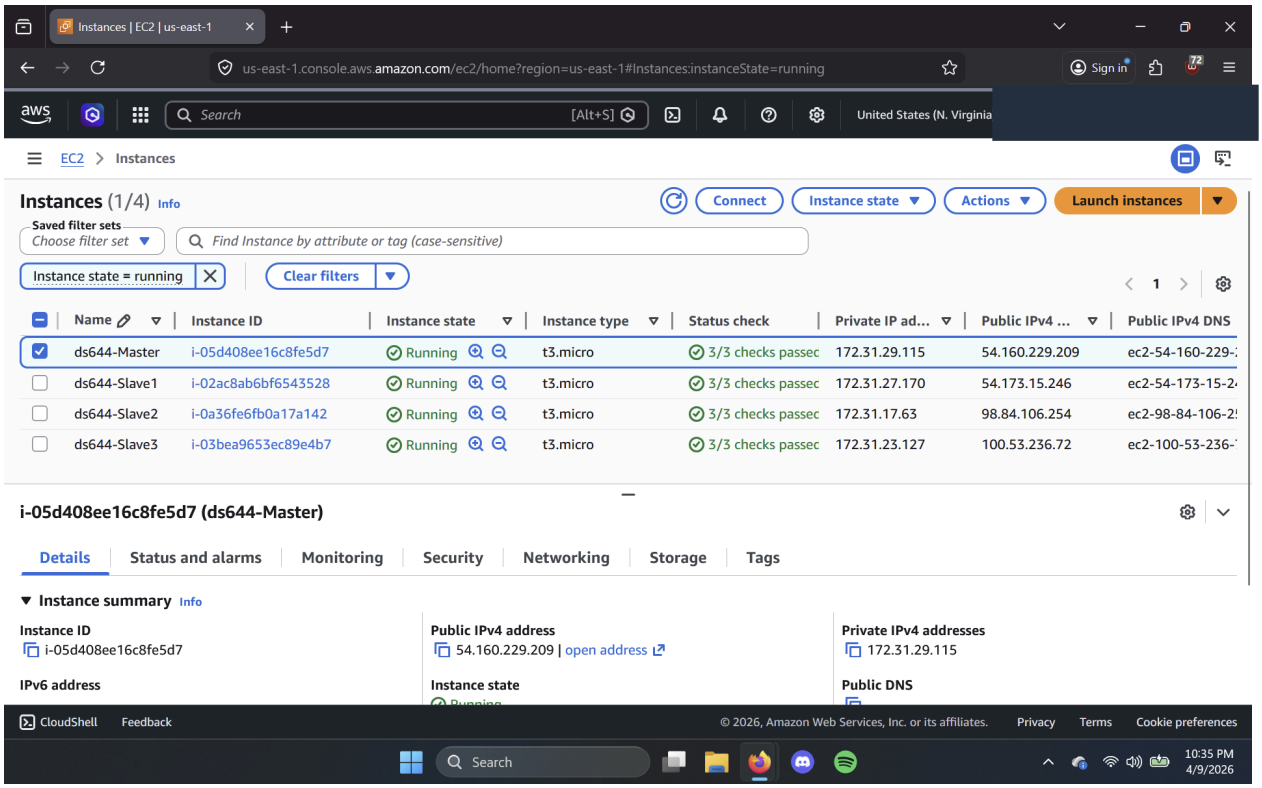
File	Size	Format
games	4.11 MB	CSV
games_details	93.08 MB	CSV
teams	4.06 KB	CSV
players	265.58 KB	CSV
ranking	15.45 MB	CSV

1.4 Reason for Selection

I chose this dataset because of the size of the dataset, its structure, and included fields. This dataset is well-suited for MapReduce processing in a fully distributed Hadoop environment due to the above factors. The games_details.csv's structure holds a player's box score statistics for each player and each of their games. This structure works well with MapReduce as in the Map phase key-value pairs can be generated and keyed by player ID and season. The value would be the associated box score statistics for each player ID / season. In the Reduce phase, the system will aggregate the necessary statistical fields to compute the desired advanced metrics: PER, Win Shares, and Estimated Plus/Minus across all games within a season. The size of this file being over 660K rows (data spanning 16 seasons and hundreds of players) justifies the use of AWS EC2 instances in fully distributed mode as sequential single-node processing would be highly inefficient to process this entire dataset. The CSV file format also is a benefit as it ensures direct compatibility with Hadoop's TextInputFormat, eliminating any need for preprocessing before ingestion into HDFS.

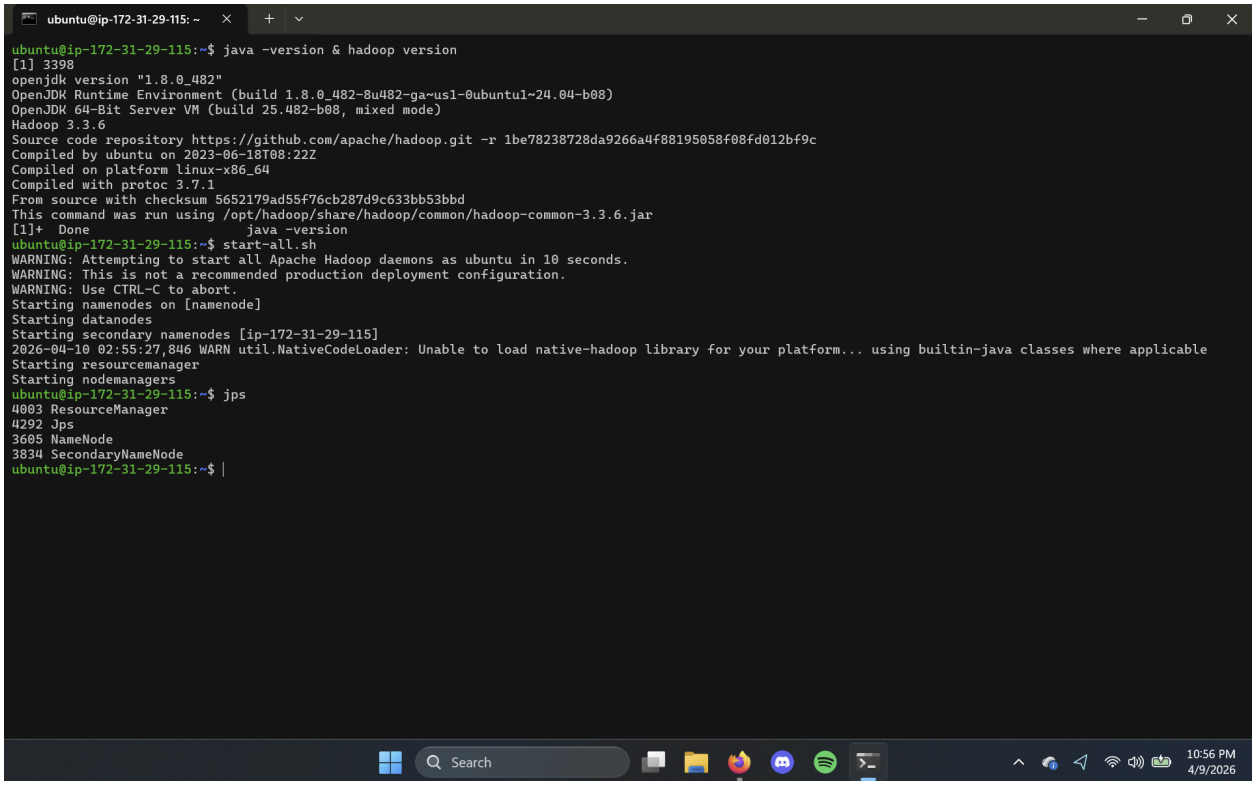
2. Environment Setup Screenshots & Explanations

2.1 Screenshot: AWS Console - 4 Ubuntu EC2 Instances Running



This screenshot shows the AWS EC2 instance dashboard. On this dashboard it shows four running t3.micro instances provisioned in the US East (N. Virginia) region. These four instances make up the Hadoop cluster for this project. This Hadoop cluster follows the standard Master-Slave architecture consisting of one master node and three slave nodes. All four instances are running with three out of three status checks passed, meaning that each instance is fully operational. Each instance is assigned both a private and public IPv4 address, with the private IP address residing within the 172.31.0.0/16 subnet. The private IP addresses allow for internal cluster communication between the Master and Slave nodes within the VPC. This is essential for Hadoop's distributed coordination and HDFS block replication across nodes. The public IP addresses enable external SSH access to each instance for configuration, monitoring, and job submission. The Master node (ds644-Master) is responsible for running Hadoop's NameNode and ResourceManager daemons which handle block metadata and coordinate scheduling and execution of MapReduce jobs across the three Slave nodes. Each slave node runs the corresponding DataNode and NodeManager daemons for distributed data storage and parallel task execution.

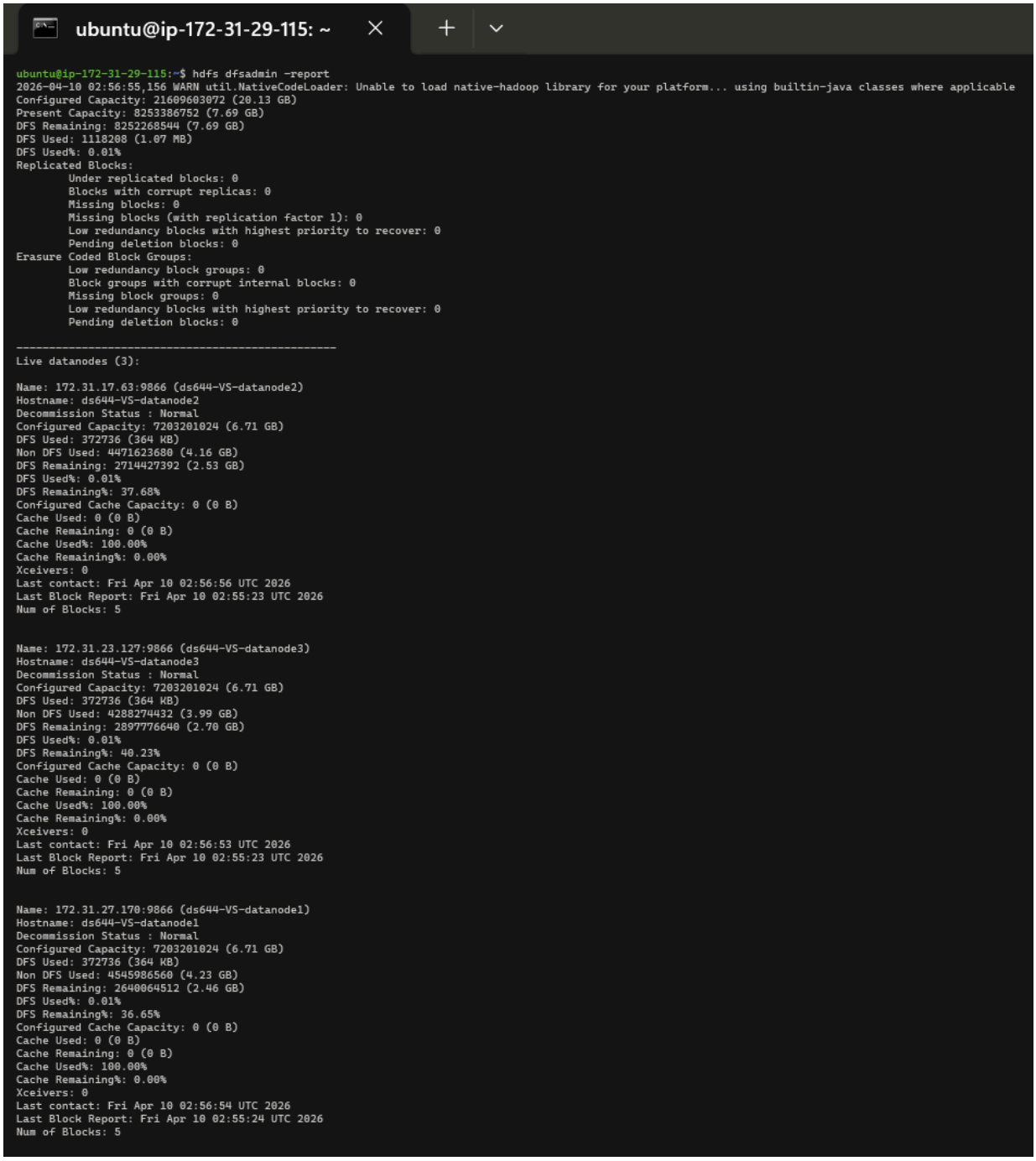
2.2 Screenshot: Java / Hadoop Installation & Hadoop Cluster



This screenshot shows the successful installation of Java and Hadoop on the Master node, as well as the startup of all required Hadoop daemons across the cluster. The output of the “java -version” command confirms that OpenJDK version 1.8.0_482 is installed. The “hadoop version” command confirms that Hadoop version 3.3.6 is installed. After verifying the installation of Java and Hadoop, the “start-all.sh” script was executed to initialize the Hadoop daemons across the cluster. The output of this execution shows the sequential startup of the NameNode, DataNodes, Secondary NameNode, ResourceManager, and NodeManagers. After the successful execution of the start up script, the “jps” command was executed to confirm that all necessary Hadoop daemons are running on the MasterNode. The daemons listed:

- NameNode (PID 3605) - manages the HDFS filesystem namespace and coordinates block locations across all DataNodes.
- Secondary NameNode (PID 3834) - backup NameNode. Periodically merges with the latest edit log to ensure the most up-to-date backup.
- ResourceManager (PID 4003) - oversees the scheduling and allocation of computational resources for MapReduce job execution.

2.3 Screenshot: Hadoop Cluster - Fully-Distributed Mode



This screenshot shows the execution and output of the “hdfs dfsadmin -report” command on the Master node. This command provides a comprehensive status report of the HDFS across the entire cluster. This screenshot confirms that the Hadoop cluster is operating in fully-distributed mode, with all Slave nodes actively participating. The command begins with an overview of the overall filesystem capacity and utilization across the entire cluster. The report then goes on to show the three live DataNodes connected to the cluster, which are the three Slave nodes

shown in Screenshot 2.1. For each Slave node it shows the capacity of the node, current usage, % remaining, # of active blocks reported, and other statistics and attributes. The execution of “hdfs dfsadmin -report” with three active DataNodes, zero corrupted or missing blocks, and all nodes reporting as healthy confirms that the Hadoop cluster is configured and operational in fully distributed mode. The environment is all set up and ready for dataset ingestion and execution of the MapReduce job.

3. MapReduce Code

3.1 Approach

The goal of this program is to compute three advanced NBA metrics, Player Efficiency Rating (PER), Win Shares (WS), and Estimated Plus/Minus (EPM), across hundreds of players and multiple seasons from a Kaggle dataset of over 600K player-game records. For the MapReduce code, the mapper would create K-V pairs of player per-game statistics in parallel, then the reducer would aggregate the per-player season-level totals and eventually the advanced metrics we are looking to calculate. The key the mapper created was a composite key of three fields: player ID, team ID, and season. The supplementary CSV files like teams.csv and ranking.csv were used and loaded into Hadoop's distributed cache and memory in the setup() method before any reducer call was made. This allowed me to be able to translate the key created from the mapper into human-readable output.

3.2 Code Explanation

Below, I will include screenshots of my code, followed by explanations / descriptions for each screenshot.

```
public static class PlayerStatsMapper extends Mapper<Object, Text, Text, Text> {

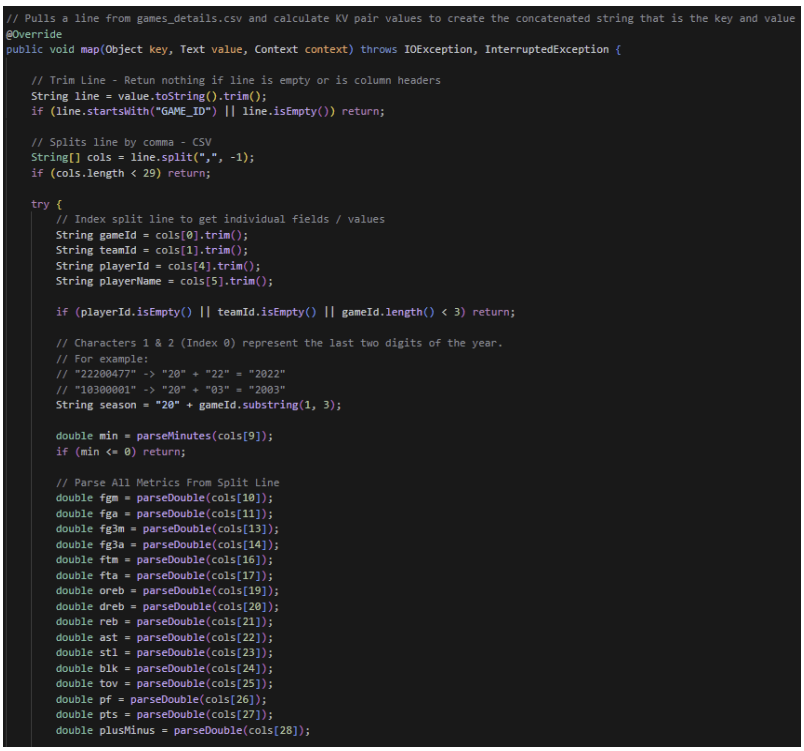
    // Initialize Output Variables
    private Text outKey = new Text();
    private Text outValue = new Text();

    // Given String of Text Containing Time - Get Minute Value
    private double parseMinutes(String s) {
        try {
            s = s.trim();
            if (s.isEmpty()) return 0.0;
            if (s.contains(":")) {
                String[] p = s.split(":");
                return Double.parseDouble(p[0]) + Double.parseDouble(p[1]) / 60.0;
            }
            return Double.parseDouble(s);
        } catch (Exception e) { return 0.0; }
    }

    // Safe parseDouble - try catch exception - give default value of 0.0
    private double parseDouble(String s) {
        try { return Double.parseDouble(s.trim()); }
        catch (Exception e) { return 0.0; }
    }
```

This screenshot includes the beginning of the PlayerStatsMapper class which extends Hadoop's built-in Mapper class. Two Text objects, outKey and outValue, are declared as variables and initialized once at a class level rather than inside the map() method. I did this because it improves performance by reusing the same instance, minimizing memory allocation overhead. If those variables were initialized in the map() method, then it would generate hundreds of thousands of objects across the entire input dataset. Below that is two private helper methods:

parseMinutes() and parseDouble(). The parseMinutes() method was created because the minutes column in the CSV file was stored in “MM:SS” format, rather than decimal format. The method would trim the input string, return 0.0 if the value is empty or contains an error, and convert “MM:SS” format to decimal format. The parseDouble() method provides a safe numeric parsing method that returns 0.0 as default. This method is helpful in handling missing, null, and non-numeric values that would otherwise cause the MapReduce job to fail or throw an error.



```java
// Pulls a line from games_details.csv and calculate KV pair values to create the concatenated string that is the key and value
@Override
public void map(Object key, Text value, Context context) throws IOException, InterruptedException {

    // Trim Line - Retun nothing if line is empty or is column headers
    String line = value.toString().trim();
    if (line.startsWith("GAME_ID") || line.isEmpty()) return;

    // Splits line by comma - CSV
    String[] cols = line.split(",", -1);
    if (cols.length < 29) return;

    try {
        // Index split line to get individual fields / values
        String gameId = cols[0].trim();
        String teamId = cols[1].trim();
        String playerId = cols[4].trim();
        String playerName = cols[5].trim();

        if (playerId.isEmpty() || teamId.isEmpty() || gameId.length() < 3) return;

        // Characters 1 & 2 (Index 0) represent the last two digits of the year.
        // For example:
        // "22200477" -> "20" + "22" = "2022"
        // "10300001" -> "20" + "03" = "2003"
        String season = "20" + gameId.substring(1, 3);

        double min = parseMinutes(cols[9]);
        if (min <= 0) return;

        // Parse All Metrics From Split Line
        double fgm = parseDouble(cols[10]);
        double fga = parseDouble(cols[11]);
        double fg3m = parseDouble(cols[13]);
        double fg3a = parseDouble(cols[14]);
        double ftm = parseDouble(cols[16]);
        double fta = parseDouble(cols[17]);
        double oreb = parseDouble(cols[19]);
        double dreb = parseDouble(cols[20]);
        double reb = parseDouble(cols[21]);
        double ast = parseDouble(cols[22]);
        double stl = parseDouble(cols[23]);
        double blk = parseDouble(cols[24]);
        double tov = parseDouble(cols[25]);
        double pf = parseDouble(cols[26]);
        double pts = parseDouble(cols[27]);
        double plusMinus = parseDouble(cols[28]);
```

This screenshot includes the beginning of the map() method of the Mapper class which is used for every line of the input CSV file. The method begins by converting the raw input value to a trimmed string. The map() returns nothing if the line read is empty or contains the column headers for the file. The method then splits the line on commas using split(“,”, -1), which the “-1” flag indicates to preserve the trailing empty fields that would normally be dropped. Inside the “try” block, four identifying fields are extracted based on their column indices: game ID, team ID, player ID, and player name. There is a check after these fields are extracted ensuring that none are empty and of the minimum length required, otherwise return nothing. The season year is the next field extracted, by pulling it from the “GAME_ID” field which contains the last two digits of the season year. Minutes played is parsed using the parseMinutes() helper method we created, and used to filter the rows by removing any where the minutes are zero or less. Using parseDouble(), the remaining statistical fields we need for the advanced metrics are parsed from the verified column indices.

```
        // Create Key
        outKey.set(playerId + "|" + teamId + "|" + season);
        // Create Value
        outValue.set(TAG_STATS + "|" + playerName + "|" + min  + "|" + fgm  + "|" + fga  + "|" + fg3m + "|" + fg3a +
                     "|" + ftm  + "|" + fta  + "|" + oreb + "|" + dreb + "|" + reb  + "|" + ast  + "|" + stl  + "|" + blk  +
                     "|" + tov  + "|" + pf   + "|" + pts  + "|" + plusMinus);

        // Output KV Pair
        context.write(outKey, outValue);

    } catch (Exception e) { /* Skips any rows with bad data */ }
```

This screenshot shows the ending of the "try" block and subsequently the ending of the map() method. After successfully parsing and validating the fields pulled from games_details.csv, the map() method ends with creating the key-value pair that will be used by the Reducer to compute the output. The output key is a composite key of the player ID, team ID, and season because it groups all games played by a specific player on a specific team within a specific season together into one single reducer call. The output value is generated in a similar way containing the player name and all 17 parsed statistical fields in a fixed order. The newly created key-value pair is then written to the MapReduce context which passes the record into the shuffle and sort phase. The "try-catch" block in the map() method is used to ignore / discard any row that raises an unexpected exception which prevents a single record from causing the entire mapper task to fail.

```
public static class MetricsReducer extends Reducer<Text, Text, Text, Text> {
    // Initialize Output Variable
    private Text outValue = new Text();

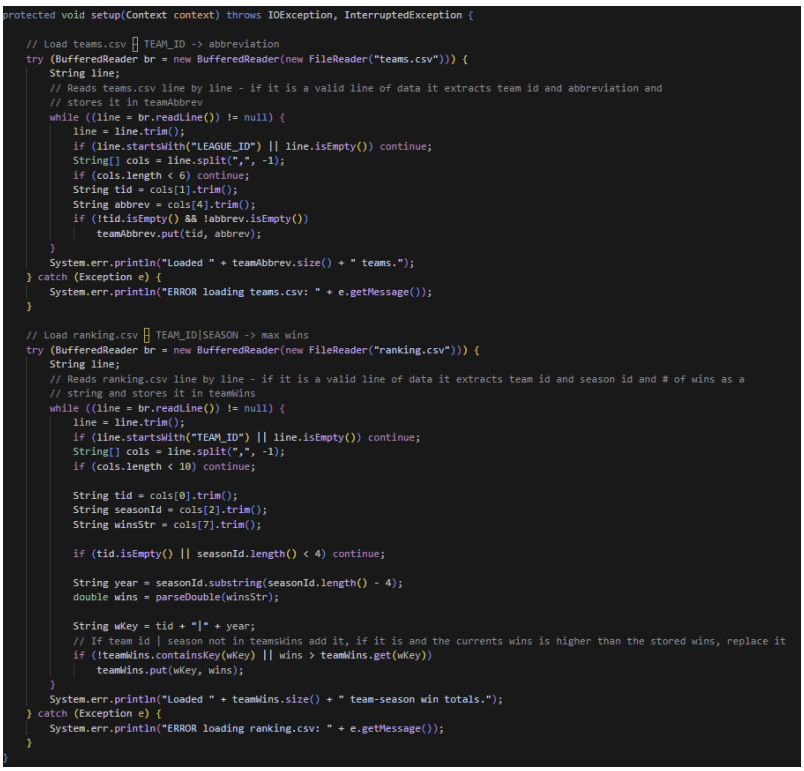
    // League-Level normalization constants used in advanced metrics
    private static final double LEAGUE_PACE_FACTOR = 15.0;
    private static final double MARGINAL_PTS_PER_WIN = 30.0;
    private static final double EPM_MINS_FACTOR = 36.0;

    // Lookup tables for team abbreviation and wins per season
    private Map<String, String> teamAbbrev = new HashMap<>();
    private Map<String, Double> teamWins = new HashMap<>();

    // Safe parseDouble - try catch exception - give default value of 0.0
    private double parseDouble(String s) {
        try { return Double.parseDouble(s.trim()); }
        catch (Exception e) { return 0.0; }
    }
```

This screenshot includes the beginning of the MetricsReducer class which extends Hadoop's built-in Reducer class. A single Text object instance is initialized at the beginning of the class for the same efficiency reason as the Mapper. Additionally, three league-level normalization constants are initialized as they will be used in the calculation of the advanced metrics. "LEAGUE_PACE_FACTOR" is a constant that represents the average PER in the NBA, which is used to scale the raw uPER calculation into a normalized value. By normalizing this metric for each player, it would allow for more meaningful cross-player comparisons on a standard scale. "MARGINAL_PTS_PER_WIN" represents the approximate number of net statistical contribution points required to generate one additional team win. This constant

is used as the denominator in the Win Shares formula. The “EPM_MINS_FACTOR” represents the standard NBA benchmark for 36 minutes to normalize the raw plus/minus differential. After, two HashMap instances are declared: “teamAbbrev” which maps team IDs to their abbreviations and “teamWins” which maps team-season combinations to win totals. These are instantiated at a class-level for the same reasons as before. The same safe parseDouble() helper method is also included to provide fault-tolerant numeric parsing.



```
protected void setup(Context context) throws IOException, InterruptedException {

    // Load teams.csv — TEAM_ID -> abbreviation
    try (BufferedReader br = new BufferedReader(new FileReader("teams.csv"))) {
        String line;
        // Reads teams.csv line by line - if it is a valid line of data it extracts team id and abbreviation and
        // stores it in teamAbbrev
        while ((line = br.readLine()) != null) {
            line = line.trim();
            if (line.startsWith("LEAGUE_ID") || line.isEmpty()) continue;
            String[] cols = line.split(",", -1);
            if (cols.length < 6) continue;
            String tid = cols[1].trim();
            String abbrev = cols[4].trim();
            if (!tid.isEmpty() && !abbrev.isEmpty())
                teamAbbrev.put(tid, abbrev);
        }
        System.err.println("Loaded " + teamAbbrev.size() + " teams.");
    } catch (Exception e) {
        System.err.println("ERROR loading teams.csv: " + e.getMessage());
    }

    // Load ranking.csv — TEAM_ID|SEASON -> max wins
    try (BufferedReader br = new BufferedReader(new FileReader("ranking.csv"))) {
        String line;
        // Reads ranking.csv line by line - if it is a valid line of data it extracts team id and season id and # of wins as a
        // string and stores it in teamWins
        while ((line = br.readLine()) != null) {
            line = line.trim();
            if (line.startsWith("TEAM_ID") || line.isEmpty()) continue;
            String[] cols = line.split(",", -1);
            if (cols.length < 10) continue;

            String tid = cols[0].trim();
            String seasonId = cols[2].trim();
            String winsStr = cols[7].trim();

            if (tid.isEmpty() || seasonId.length() < 4) continue;

            String year = seasonId.substring(seasonId.length() - 4);
            double wins = parseDouble(winsStr);

            String wKey = tid + "|" + year;
            // If team id | season not in teamsWins add it, if it is and the currents wins is higher than the stored wins, replace it
            if (!teamWins.containsKey(wKey) || wins > teamWins.get(wKey))
                teamWins.put(wKey, wins);
        }
        System.err.println("Loaded " + teamWins.size() + " team-season win totals.");
    } catch (Exception e) {
        System.err.println("ERROR loading ranking.csv: " + e.getMessage());
    }
}
```

This screenshot includes the setup() method which pre-loads two reference files (teams.csv and ranking.csv) from the DistributedCache into memory before any reduce calls are made. The first code block reads teams.csv line by line, skipping the header row and any empty lines. Each line is then split by comma, where the team ID and abbreviation are then extracted from this split string based on pre-determined column indices. Only rows with non-empty values are inserted into the “teamAbbrev” HashMap. The entire code block is wrapped in a “try-catch” so that if teams.csv cannot be found or read the code still continues rather than crashing. The second code block reads ranking.csv line by line, following the same steps. Each line is validated to ensure it contains the correct number of fields and are properly populated. The team ID, season identifier, and win total are extracted from ranking.csv based on the column indices. A composite key of team ID and season is created and used to populate the “teamWins” HashMap. Since ranking.csv contains daily standings entries, a conditional check was added, “teamWins.containsKey(wKey) || wins > teamWins.get(wKey)” that ensures only the highest win total observed for each team-season combo is kept and stored in the HashMap, otherwise overwrite it.



```java
public void reduce(Text key, Iterable<Text> values, Context context) throws IOException, InterruptedException {

    // Read Key and Split by | - Format: PLAYER_ID|TEAM_ID|SEASON
    String keyStr = key.toString();
    String[] parts = keyStr.split("\\|", 3);
    if (parts.length < 3) return;

    // Store Key Parts into Variables
    String playerId = parts[0];
    String teamId = parts[1];
    String season = parts[2];

    // Initialize Variables to Store Season Totals & Player Name (Player_<playerId>)
    double totalMin = 0, totalFGM = 0, totalFGA = 0;
    double totalFG3M = 0, totalFG3A = 0, totalFTM = 0, totalFTA = 0;
    double totalOREB = 0, totalDREB = 0, totalREB = 0, totalAST = 0;
    double totalSTL = 0, totalBLK = 0, totalTOV = 0, totalPF = 0;
    double totalPTS = 0, totalPlusMinus = 0;
    double gamesPlayed = 0;
    String playerName = "Player_" + playerId;
    // Loops through each value (game total/record) for each key
    for (Text val : values) {
        // Convert Value to String - Split & Extract Values Into Running Total
        String v = val.toString();
        String[] p = v.split("\\|", -1);
        if (p.length < 19 || !p[0].equals(TAG_STATS)) continue;

        if (!p[1].isEmpty()) playerName = p[1];

        totalMin += parseDouble(p[2]);
        totalFGM += parseDouble(p[3]);
        totalFGA += parseDouble(p[4]);
        totalFG3M += parseDouble(p[5]);
        totalFG3A += parseDouble(p[6]);
        totalFTM += parseDouble(p[7]);
        totalFTA += parseDouble(p[8]);
        totalOREB += parseDouble(p[9]);
        totalDREB += parseDouble(p[10]);
        totalREB += parseDouble(p[11]);
        totalAST += parseDouble(p[12]);
        totalSTL += parseDouble(p[13]);
        totalBLK += parseDouble(p[14]);
        totalTOV += parseDouble(p[15]);
        totalPF += parseDouble(p[16]);
        totalPTS += parseDouble(p[17]);
        totalPlusMinus += parseDouble(p[18]);
        gamesPlayed++;
    }

    if (totalMin <= 0 || gamesPlayed == 0) return;

    // Pull team abbreviation and wins from HashMaps populated in setup()
    String displayTeam = teamAbbrev.getOrDefault(teamId, teamId);
    double wins = teamWins.getOrDefault(teamId + "|" + season, 0.0);
```

This screenshot includes the beginning of the reduce() method. The method starts by converting the incoming key to a string and splitting on “|” to extract player Id, team Id, and season. If the key is missing one of these three, the record is skipped. Each of the statistical fields we will be using are initialized to zero along with the default for “playerName”: “Player_” concatenated with the player Id. The method iterates over every value in the group, which represents one individual game played by this player on this team in this season. Just like the key, the value is delimited by “|” which is used to extract the individual values later on. The player’s name and statistical fields are parsed (some using parseDouble()) and added to a running total for that key. Once the loop is completed, the variables initialized would contain the player’s full season statistics on that team. Using the HashMaps created in the setup() method, both the team abbreviation and win total are resolved for each of the records.

```
// Pull team abbreviation and wins from HashMaps populated in setup()
String displayTeam = teamAbbrev.getOrDefault(teamId, teamId);
double wins = teamWins.getOrDefault(teamId + "|" + season, 0.0);

// Advanced Metric 1: Player Efficiency Rating (PER)
// Weighted per-minute production normalized to league average of 15
double uPER = (1.0 / totalMin) * (totalPTS + (0.4 * totalFGM) - (0.7 * totalFGA) - (0.4 * (totalFTA - totalFTM)) + (0.7 * totalOREB)
            + (0.3 * totalDREB) + totalSTL + (0.7 * totalAST) + (0.7 * totalBLK) - (0.4 * totalPF) - totalTOV);
// Scale raw PER with NBA league average of 15
double per = uPER * LEAGUE_PACE_FACTOR;

// Advanced Metric 2: Win Shares (WS)
// Net positive contributions / marginal points per win (~30)
double netContrib = totalPTS + totalREB + totalAST + totalSTL + totalBLK - totalTOV - (totalFGA - totalFGM) - (totalFTA - totalFTM);
double winShares  = netContrib / MARGINAL_PTS_PER_WIN;
if (wins > 0 && winShares > wins) winShares = wins;
if (winShares < 0) winShares = 0;

// Advanced Metric 3: Estimated Plus/Minus (EPM)
// Raw plus/minus normalized to per-36-minute rate
double epm = (totalPlusMinus / totalMin) * EPM_MINS_FACTOR;

// Format and generate output
String outputKey = playerId + "|" + teamId + "|" + season;
String output = String.format(
    "SEASON: %s | PLAYER: %-25s | TEAM: %-5s | GP: %3.0f | MIN: %6.1f | PER: %6.2f | WIN_SHARES: %5.2f | EPM: %+6.2f",
    season, playerName, displayTeam, gamesPlayed, totalMin, per, winShares, epm
);

// Output Key - Value Created Above
outValue.set(output);
context.write(new Text(outputKey), outValue);
```

This screenshot includes the remainder of the reduce() method after the season totals are calculated. Here is where the Reducer computes the three advanced metrics. PER is calculated by first computing the raw uPER by multiplying “1 / totalMin” by a weighted sum of the statistical contributions. Then using this raw uPER it is scaled by the “LEAGUE_PACE_FACTOR”, resulting in the final PER value. The weights used in this formula are based on their relative value to team performance. This calculation would result in a PER value that we could interpret to estimate the skill and impact of a player. If the PER is below 15, then it is a below-average player, while above 15 would indicate an above average player. Win Shares are calculated by first calculating a net contribution value and dividing it by the “MARGINAL_PTS_PER_WIN” constant. After this is calculated two checks are ran, one compares the team’s season win total to the Win Shares, if the Win Shares is greater it is capped at the team’s actual win total. The other checks to see if the Win Shares is negative, if so, cap it at 0. The Estimated Plus/Minus is calculated by dividing the cumulative raw plus/minus differential by total minutes and then multiplying it by the “EPM_MINS_FACTOR” to get a per-36-minute rate. The output of the reduce() method is formatted using “String.format()” with fixed-width specifiers and all numeric values formatted to two decimal places. The key and output are both stored in Text objects and written to the MapReduce context.

```java
public static void main(String[] args) throws Exception {

    // Less than 2 arguments - end execution and output help
    if (args.length < 2) {
        System.err.println("Usage: NBAAdvancedMetrics <games_details> <output>");
        System.exit(1);
    }

    // Initialize Hadoop Configuration and create new MapReduce job instance
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "NBA Advanced Metrics: PER, Win Shares, EPM");

    // Set the main class for JAR file lookup on the datanodes
    // Set Reduce class and define output key and value data types
    job.setJarByClass(NBAAdvancedMetrics.class);
    job.setReducerClass(MetricsReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(Text.class);

    // Add teams.csv and ranking.csv files to DistributedCache for setup()
    job.addCacheFile(new URI("hdfs:///nba/input/teams/teams.csv#teams.csv"));
    job.addCacheFile(new URI("hdfs:///nba/input/ranking/ranking.csv#ranking.csv"));

    // Set the Input Path (Multiple) for the newly created MapReduce job instance to the file path for games_details.csv,
    // the TextInputFormat class, and the Mapper class
    MultipleInputs.addInputPath(job, new Path(args[0]), TextInputFormat.class, PlayerStatsMapper.class);

    // Set the Output Path for the newly created MapReduce job instance to the output file path passed as an argument
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

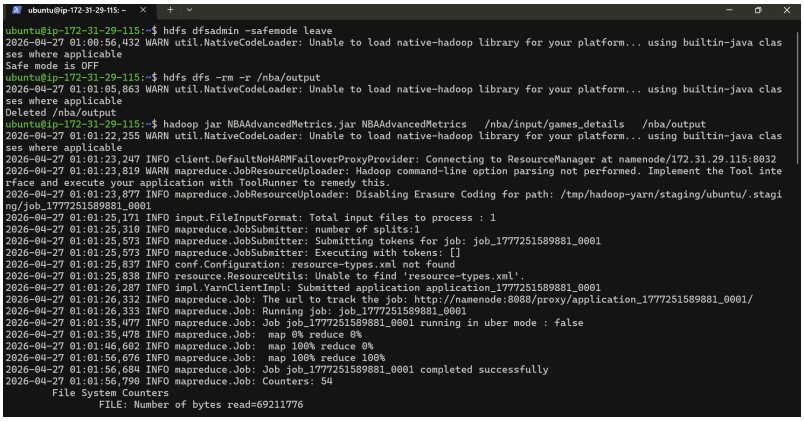
    // If job completes successfully exit program
    boolean success = job.waitForCompletion(true);
    System.exit(success ? 0 : 1);
}
```

This screenshot includes the main() method which configures and submits the MapReduce job to the Hadoop cluster. It first checks the provided arguments, if fewer than two are received, then it exits the program and outputs an error message. A job object instance is created with a display name of “NBA Advanced Metrics: PER, Win Shares, EPM”, then using the setJarByClass() method we indicate to Hadoop which JAR file to distribute to the cluster nodes. The Reducer class as well as the data type of the output key and value are also set in this code block. The two lookup files are then added to Hadoop's DistributedCache using addCacheFile() and their HDFS file paths. The input paths for the MapReduce job are set using MultipleInputs.addInputPath(), which sets the input file directory and related Java classes. The output path for the MapReduce job is set from the second argument passed using FileOutputFormat.setOutputPath(). The main() method then ends with the job.waitForCompletion(true) line which submits the job to the cluster and blocks until it finishes. This program completes and outputs a status code of 1 or 0 indicating success or failure.

3.3 Compilation & Execution Commands

```
ubuntu@ip-172-31-29-115: ~
ubuntu@ip-172-31-29-115:~$ ls
NBAAdvancedMetrics.jar   games_details.csv  mapreduce          nba_results.txt  ranking.csv
NBAAdvancedMetrics.java  job_output.log     nba_MRresults.txt  players.csv      teams.csv
ubuntu@ip-172-31-29-115:~$ export HADOOP_CLASSPATH=$(hadoop classpath)
ubuntu@ip-172-31-29-115:~$ javac -classpath $HADOOP_CLASSPATH -d . NBAAdvancedMetrics.java
ubuntu@ip-172-31-29-115:~$ ls *.class
'NBAAdvancedMetrics$MetricsReducer.class'  'NBAAdvancedMetrics$PlayerStatsMapper.class'   NBAAdvancedMetrics.class
ubuntu@ip-172-31-29-115:~$ jar -cvf NBAAdvancedMetrics.jar *.class
added manifest
adding: NBAAdvancedMetrics$MetricsReducer.class(in = 6498) (out= 3228)(deflated 50%)
adding: NBAAdvancedMetrics$PlayerStatsMapper.class(in = 3254) (out= 1532)(deflated 52%)
adding: NBAAdvancedMetrics.class(in = 2069) (out= 1073)(deflated 48%)
ubuntu@ip-172-31-29-115:~$ ls
'NBAAdvancedMetrics$MetricsReducer.class'      NBAAdvancedMetrics.jar    job_output.log      nba_results.txt   teams.csv
'NBAAdvancedMetrics$PlayerStatsMapper.class'   NBAAdvancedMetrics.java   mapreduce           players.csv
 NBAAdvancedMetrics.class                      games_details.csv         nba_MRresults.txt   ranking.csv
ubuntu@ip-172-31-29-115:~$
```

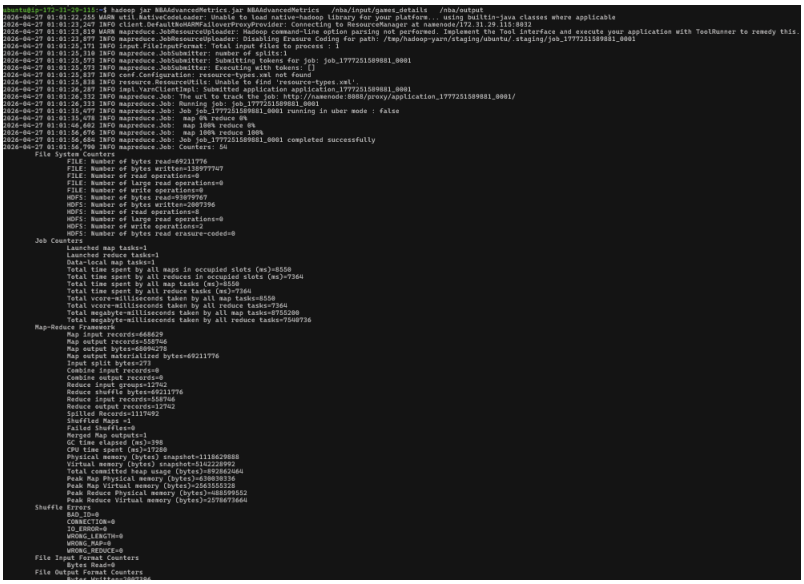
- export HADOOP_CLASSPATH=$(hadoop classpath) - Sets the HADOOP_CLASSPATH environment variable to the full list of Hadoop library JAR paths. This makes those Hadoop libraries available to the Java compiler so Hadoop-specific classes such as Mapper, Reducer, and Job can be used without error.
- javac -classpath $HADOOP_CLASSPATH -d . NBAAdvancedMetrics.java - Compiles NBAAdvancedMetrics.java using the Java compiler with the Hadoop classpath, outputting the compiled .class files into the current directory.
- jar -cvf NBAAdvancedMetrics.jar *.class - Packages all compiled .class files into a single executable JAR file named NBAAdvancedMetrics.jar.



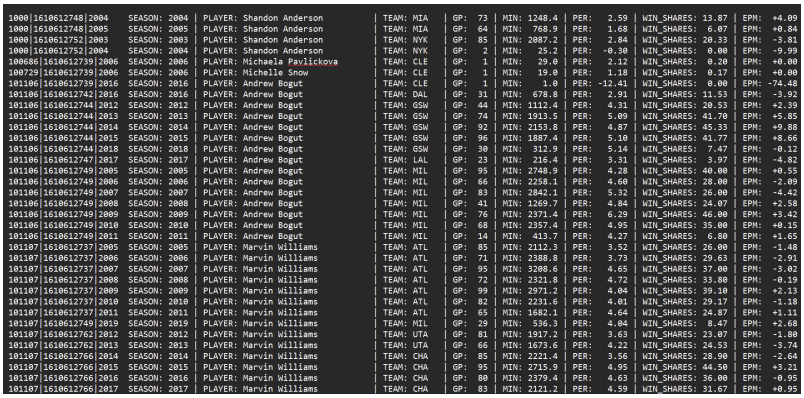
- hdfs dfsadmin -safemode leave - Takes the HDFS NameNode out of safe mode, which it enters automatically on cluster restart while verifying filesystem integrity.
- hdfs dfs -rm -r /nba/output - Recursively deletes the existing /nba/output directory from HDFS. Hadoop requires the output directory to not exist before a job is submitted.
- hadoop jar NBAAdvancedMetrics.jar NBAAdvancedMetrics /nba/input/games_details /nba/output - Submits the MapReduce job to the Hadoop cluster, specifying the JAR file, the main class to execute, the HDFS input directory containing games_details.csv, and the HDFS output path where results will be written.

4. Code Execution & Output Interpretation

4.1 MapReduce Job Completion



This screenshot shows the successful execution of the NBAAdvancedMetrics MapReduce job on the Hadoop cluster. The job was submitted using the hadoop jar command and was assigned job ID job_1777251589881_0001. You can see the job ran in fully distributed mode by the uber mode being set to false. The job progressed from 0% complete for both map and reduce phases to both 100% completion before completing successfully. The remainder of the output for the execution command provides a detailed breakdown of the execution. Under the File System Counters, HDFS read 93,079,767 bytes of input, which includes games_details.csv, teams.csv, and ranking.csv. The MapReduce program also wrote 13,897,747 bytes to HDFS as the final output. Under the Map-Reduce Framework, the Mapper processed all 668,629 input records from games_details.csv and outputted 558,746 records. The difference in the number of input and output records for the Mapper phase can be tied to the filtering out of records with empty values or where the player did not play at least a minute in the game. The Reducer received these 558,746 records grouped into 12,742 unique PLAYER_ID|TEAM_ID|SEASON key groups. The Reducer then produced 12,742 output records, one per unique key group, each containing the computed PER, Win Shares, and EPM for that player on that team in that season. The File Output Format Counters confirm that 2,007,396 bytes were written to the /nba/output directory in HDFS as the final output directory.



This screenshots shows a portion of the MapReduce job's output. It includes the computed advanced metrics for multiple players across multiple seasons. Each row represents one unique player-team-season combination and is structured as follows: “PLAYER_ID|TEAM_ID|SEASON”. After the key, it is followed by the player's name and a row of tab-separated formatted values, one for each of the computed fields. Each row contains the season year, player name, team abbreviation, games played, total minutes played, Player Efficiency Rating, Win Shares, and Estimated Plus/Minus. One of the interesting things about this output is that players who appeared on multiple teams across their careers have a separate row for each team-season combination. Overall, the text file output shows that the MapReduce job successfully processed all 668,629 input records, correctly grouped them, and produced analytically meaningful advanced metrics for each combination.

5. Challenges & Troubleshooting

5.1 Challenges Faced

1. DataNode and NodeManager Instability

   Even after successful Hadoop cluster set up and confirming SSH connectivity was established between the nodes in the cluster, individual slave nodes would repeatedly drop out of the cluster or would be unreachable during job execution. This issue appeared as “All datanodes are bad. Aborting” errors during the HDFS write operations or task timeout failures during the shuffle phase. The ResourceManager would be unable to reach NodeManagers on the slave nodes to clean up completed containers. For this issue, I identified that the “t3.micro” instance type (used for all nodes) contributed to this issue, as each instance only had 1 GB of RAM to run the DataNode, NodeManager, and MapReduce container processes.
2. Season Key Join Failure

   Previous iterations of my code attempted to join the player statistics from games_details.csv with the player name data from players.csv using a shared key in order to transform the output to a human readable format.

This join failed consistently as it used the “TEAM_ID” stored in players.csv, which contains the player’s most recent team, while games_details.csv contained the “TEAM_ID” of the team the player was actually on at the time of the game. Players who changed teams, even once, would have mismatched team Ids between the files, causing the join failure. I addressed this issue by reading the player’s name directly from games_details.csv rather than players.csv, eliminating the need for the join.

3. Reducer Cache Timing - Sort Order Conflict
   A previous iteration of my code tried to load the team abbreviations and win totals from teams.csv and ranking.csv through the MapReduce shuffle pipeline. However, this approach caused issues because Hadoop sorts all intermediate keys alphabetically before delivering them to the Reduce. Since the player IDs are small numeric values and the team IDs are large numeric values, the player stat records arrived in the Reduce before any team lookup records. This meant that when the advanced metric computation started the HashMaps used for looking up team abbreviations and win totals was empty, resulting in zero output records being generated.

5.2 Troubleshooting Steps

1. DataNode and NodeManager Instability
   Individual slave nodes that dropped out of the cluster or were unreachable through SSH from the other nodes were recovered through a couple different methods. First, I restarted each of the unreachable nodes using the AWS EC2 instance dashboard. Then, I would SSH into each slave node and individually restarted the DataNode and NodeManager daemons using: “hadoop-daemon.sh start datanode” and “yarn-daemon.sh start nodemanager”. SSH connectivity was then rechecked by SSHing into the nodes from my local workstation and from the NameNode as well as “yarn node -list” to verify all NodeManagers are active and running.
2. Season Key Join Failure
   The join failure was identified by inspecting sample records from the games_details.csv and players.csv files to compare the “TEAM_ID” values for the same players. Through this inspection, I confirmed the team IDs do not match for players that have been traded, meaning the join I was trying to do would not be reliable. Additionally, through this inspection it was determined the player name in games_details.csv would be used instead, eliminating the need for a join entirely.
3. Reducer Cache Timing - Sort Order Conflict
   This sort order cache conflict was identified by inspecting the “PLAYER_ID” and “TEAM_ID” values across the dataset. Through this inspection, I found that player IDs were numerically smaller than team IDs by a wide margin, player IDs are as low as 15 and team IDs start in the millions. When these two fields are sorted, player stat records would arrive to the Reducer before the lookup records identified by the team ID. To

address this issue, I moved teams.csv and ranking.csv out of the MapReduce pipeline and loaded them into the Reducer phase via Hadoop's DistributedCache in the Reducer's setup() method. The setup() method is executed once before any reduce() call, ensuring that for each reduce() call the files in the DistributedCache are pulled into memory for use. This ensures that when the advanced metrics calculations are executed that both lookup tables, "teamAbbrev" and "teamWins", were both fully populated.

5.3 Performance Observations

The primary performance bottleneck I faced through the course of this project was the "t3.micro" instance type used for all four cluster nodes. Each of these instances provide only 1 vCPU and 1 GB of RAM which constrains the cluster's ability to run concurrent Map and Reduce tasks. This forced the job to be executed with a single Map split and a single Reduce task rather than distributing the workload in parallel. To address this bottleneck, I upgraded the instance type of all cluster nodes from "t3.micro" to "t3.medium", which would add additional RAM and vCPUs and allow for multiple concurrent containers per node, enabling true parallel execution across the slave nodes. Another area of performance optimization I implemented was declaring the output key and value Text instances at a class level rather than inside the map() method. By reusing objects instead of creating new ones, it would reduce the garbage collection pressure across the over 600K map() calls.

6. Summary & Key Learnings

6.1 Project Reflection

Big Data Project - Option 1 encompassed every layer of a real-world distributed data processing pipeline. It began with provisioning and configuring a multi-node Hadoop cluster on AWS EC2 ubuntu instances. Then it led into designing and debugging a MapReduce program against a large, real-world dataset. The most significant takeaway for me was that distributed computing introduces a new class of problems that do not exist in single-node environments. Some of these issues included key sort-order conflicts, inter-node SSH connectivity failures, NodeManager instability, and the shuffle phase network dependency between nodes. Each of these issues required a deeper understanding of how Hadoop's cluster architecture operates, not just how to write a MapReduce program. Another important takeaway was the importance of verifying data and assumptions made about the data before writing code. For this project, I had to understand how the input files and datasets are structured in order to write efficient MapReduce code and reduce the amount of time I spend on debugging preventable issues. For example, if I assumed the column structure of games_details.csv without first inspecting the actual file, it would have led to incorrect column indices and an incorrect minutes format. Finally, the project also showed how a good key structure is central to a successful

MapReduce job. Through various iterations of the composite key used in my program, from PLAYER_ID|SEASON to PLAYER_ID|TEAM_ID to the final PLAYER_ID|TEAM_ID|SEASON, it made me understand how the grouping key directly determines what gets aggregated together in the Reducer. To me, this emphasizes that a poorly chosen key can produce incorrect or empty results without raising any errors.

6.2 Real-World Application

The MapReduce program I created, even though it is for NBA player advanced metrics, includes principles and processes that could be applied to many industries. Any industry where there are large volumes of event-level records that need to be aggregated into meaningful per-entity values would require the same principles and approaches to be applied in its development. In the sports analytics industry specifically, platforms such as ESPN, NBA.com, and NBA teams process hundreds of millions of tracking events and box score records per season to compute the advanced metrics that are used in decisions regarding player valuation, contract negotiations, draft selections, and in-game strategy. The pattern used in my program, generating per-event key-value pairs keyed by the player and season and then aggregating into season totals, can be applied to financial services. In financial services, for example, transaction-level records are aggregated per customer per period to detect fraud patterns or compute credit risk scores. In the digital advertising industry, click and impression events are aggregated per campaign per day to compute performance metrics such as click-through rate and cost per acquisition. In the healthcare industry, records are aggregated per patient to compute longitudinal health metrics and risk scores. Hadoop MapReduce, and similar tools such as Apache Spark, provide the infrastructure to execute this pattern consistently at scale across a distributed environment.

6.3 Future Improvements

I think there are several improvements and optimizations that could be applied to improve both the accuracy and the performance of my MapReduce program. From an accuracy standpoint, the Win Shares formula used here is an approximation that does not account for team pace, league-average offensive and defensive ratings, or positional adjustment factors that are incorporated in the official NBA Win Shares calculation. To calculate the official NBA Win Shares value for each player it would require joining additional league-level statistics into the pipeline. The current EPM calculation is also an approximation of the true Estimated Plus/Minus which uses regularized regression across all player lineups to isolate individual player impact. This calculation would require a multi-job pipeline where the first job aggregates lineup-level plus/minus data and the second job uses those results to compute player-level impact estimates. From a performance standpoint, implementing a Combiner class that performs partial aggregation of stat totals on each Mapper node before the shuffle phase would significantly reduce the volume of data transferred across the network. Finally, I would use Apache Spark instead of Hadoop MapReduce in a future implementation. This is

because Apache Spark would provide substantially better performance through its in-memory processing, and would simplify the join logic by allowing DataFrames from all five CSV files to be joined directly using Spark SQL.